UNIVERSIDADE FEDERAL DE OURO PRETO
CAMPUS MORRO DO CRUZEIRO – OURO PRETO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Ana Júlia Barros Araújo

Simulador de Campanhas Militares em C++

OURO PRETO
22 de outubro de 2026

Ana Júlia Barros Araújo

Simulador de Campanhas Militares em C++

Trabalho apresentado ao curso de Bacharelado em Ciência da Computação da Universidade Federal de Ouro Preto como requisito para avaliação.

OURO PRETO
22 de outubro de 2026

Sumário

1 Introdução **2**

1.1 Objetivo Geral 2

1.2 Objetivos Específicos 2

1.3 Justificativa 2

2 Desenvolvimento **3**

2.1 Especificação de Requisitos 3

2.2 Diagrama de Classes 4

2.3 Diagrama de Entidade Relacionamento 5

3 Conclusão **7**

1 Introdução

Este trabalho apresenta o desenvolvimento de um simulador de campanhas militares em C++, construído com os conceitos de Programação Orientada a Objetos: encapsulamento, herança, polimorfismo, classes abstratas, composição e agregação. O sistema modela exércitos formados por unidades de combate (infantaria, veículos e aeronaves), que se enfrentam em batalhas agrupadas em uma campanha. Ao final, o programa gera uma tabela de posições dos exércitos e identifica a unidade mais destrutiva.

1.1 Objetivo Geral

Desenvolver, em C++, um simulador de campanhas militares que aplique os princípios da Programação Orientada a Objetos para modelar unidades, exércitos, batalhas e campanhas, e produzir resultados estatísticos das batalhas simuladas.

1.2 Objetivos Específicos

Entrando em mais detalhes, os objetivos técnicos deste trabalho envolvem:

- Modelar a classe abstrata Unidade e suas especializações Infantaria, Veiculo e Aeronave, usando herança e polimorfismo.
- Calcular o poder de ataque de cada tipo de unidade por fórmulas próprias, com atributos gerados aleatoriamente dentro de faixas definidas.
- Compor exércitos de cinco unidades (duas de infantaria, dois veículos e uma aeronave) e confrontá-los em batalhas datadas.
- Gerar a tabela de posições dos exércitos (vitórias, empates e derrotas), na tela ou em arquivo de texto.
- Identificar e exibir a unidade mais destrutiva da campanha.

1.3 Justificativa

O domínio de um simulador de batalhas se presta bem à prática da orientação a objetos. Há uma hierarquia natural (tipos de unidades com comportamentos distintos para uma mesma operação), relações de composição claras (campanha, batalha, exército, unidade) e regras de negócio que exigem encapsulamento do estado de cada objeto. O trabalho permite exercitar esses conceitos em um problema concreto, além de gerenciamento dinâmico de memória, uso de contêineres da STL (std::vector), geração de números pseudoaleatórios e manipulação de arquivos.

2 Desenvolvimento

Neste capítulo são apresentados os requisitos do sistema, a modelagem das classes e as decisões de implementação. O código foi organizado em pares de arquivos .h/.cpp, um por classe, mais o arquivo main.cpp.

2.1 Especificação de Requisitos

Requisitos funcionais:

- **Unidade:** o sistema deve representar três tipos de unidade, todas com poder de ataque base e contador de destruições.
- **Infantaria:** possui força e velocidade. Poder base entre 30 e 60, força entre 20 e 50 e velocidade entre 10 e 40. O poder de ataque é ((poderAtaque*5) + (forca*3) + (velocidade*2)) / 10.
- **Veiculo:** possui blindagem e potência de fogo. Poder base entre 40 e 80, blindagem entre 30 e 70 e potência de fogo entre 20 e 50. O poder de ataque é ((poderAtaque*5) + (blindagem*4) + (potenciaDeFogo*1)) / 10.
- **Aeronave:** possui manobrabilidade e alcance. Poder base entre 50 e 90, manobrabilidade entre 30 e 70 e alcance entre 20 e 50. O poder de ataque é ((poderAtaque*5) + (manobralidade*3) + (alcance*2)) / 10.
- **Exército:** possui nome, lista de unidades e contadores de vitórias, derrotas e empates. Cada exército é composto por 5 unidades: 2 de infantaria, 2 veículos e 1 aeronave. Seu poder de ataque total é a soma dos poderes de suas unidades.
- **Batalha:** ocorre em uma data entre dois exércitos. Em cada ataque, compara-se o poder total dos exércitos: o de maior poder vence, e o outro é derrotado; poderes iguais resultam em empate para ambos. Somente o exército atacante, se vencer, soma uma destruição a cada uma de suas unidades.
- **Campanha:** agrega várias batalhas. Deve permitir adicionar batalhas (criando automaticamente os exércitos e suas unidades), simulá-las, gerar a tabela de posições e mostrar a unidade mais destrutiva.
- **Data:** a data da batalha é representada por dia, mês e ano, com formatação no padrão dd/mm/aaaa.

Requisitos não funcionais:

- Implementação em C++ (padrão C++11 ou superior), com separação em arquivos de cabeçalho e implementação.

- Atributos privados com acesso por métodos get/set (encapsulamento).
- Geração de valores aleatórios
- Liberação de memória dinâmica nos destrutores (Campanha → Batalha → Exército → Unidade).

2.2 Diagrama de Classes

As classes a seguir foram divididas de tal forma a facilitar e deixar o mais claro possível o entendimento a estrutura da organização do projeto.

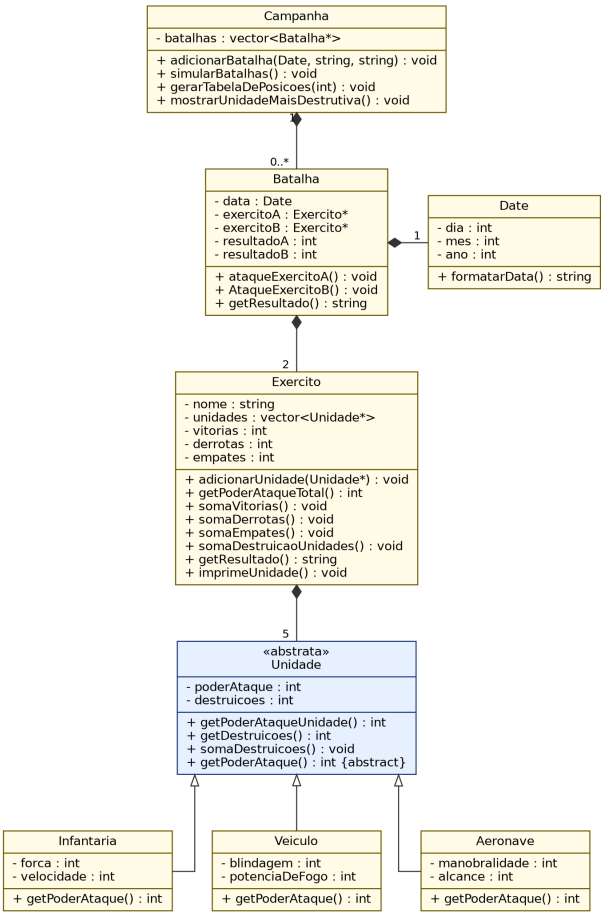


Figura 1: Diagrama de Classes

A figura 1 apresenta as classes do sistema e seus relacionamentos. Cada classe representa:

- **Unidade:** classe abstrata base. Guarda poderAtaque e destruicoes, e declara o método virtual puro getPoderAtaque(), que cada subclasse implementa com sua própria fórmula (polimorfismo). Também oferece somaDestruicoes().

- **Infantaria, Veiculo e Aeronave:** subclasses concretas de Unidade. Cada uma define seus atributos específicos, sorteados no construtor dentro das faixas do requisito, e sobrescreve getPoderAtaque().

- **Exército:** compõe cinco unidades (vector<Unidade*>). Mantém nome e placar, e oferece adicionarUnidade(), getPoderAtaqueTotal(), somaVitorias(), somaDerrotas(), somaEmpates(), somaDestruicaoUnidades(), getResultado() e imprimeUnidade(). Por tratar a lista como polimórfica, soma o poder de ataque sem conhecer o tipo concreto de cada unidade.

- **Date:** guarda dia, mês e ano, e formata a data como texto. Compõe uma Batalha.

- **Batalha:** composta por uma Date e dois Exercito (A e B). Os métodos ataqueExercitoA() e AtaqueExercitoB() comparam os poderes totais e atualizam vitórias, derrotas, empates e destruições. getResultado() retorna o placar textual, por exemplo, "A 1 X 0 B".

- **Campanha:** composta por várias Batalha (vector<Batalha*>). adicionarBatalha() monta os dois exércitos com suas cinco unidades; simularBatalhas() executa os ataques de A e de B em cada batalha; gerarTabelaDePosicoes() consolida e ordena os exércitos; mostrarUnidadeMaisDestrutiva() varre as unidades procurando o maior número de destruições.

Relacionamento: Infantaria, Veiculo e Aeronave herdam de Unidade (generalização). Exercito compõe 5 Unidade, Batalha compõe 2 Exercito e 1 Date, e Campanha compõe 0..* Batalha. Como cada classe possui a posse e o destrutor libera os objetos agregados, todos são relacionamentos de composição.

2.3 Diagrama de Entidade Relacionamento

Não se aplica. O sistema não utiliza banco de dados: todos os dados são mantidos em memória durante a execução, e a única persistência é a gravação opcional da tabela de posições no arquivo tabela.txt.Não se aplica. O sistema não utiliza banco de dados: todos os dados são mantidos em memória durante a execução, e a única persistência é a gravação opcional da tabela de posições no arquivo tabela.txt.

A função main cria uma data e uma Campanha, adiciona duas batalhas (exércitos "1"x "2"e "3"x "4"), simula as batalhas imprimindo o placar de cada uma, gera a tabela de posições e exibe a unidade mais destrutiva. Em cada batalha, o ataque de A é resolvido e depois o de B, o que permite que ambos os lados somem destruições conforme o resultado de cada ataque.

- **Resultado dos ataques:** como o poder das unidades não muda entre o ataque de A e o de B, a comparação é a mesma nos dois ataques. Assim, o exército mais forte vence nos dois, e o placar acumula duas vitórias para ele (ou dois empates).
- **Destruições:** a destruição é somada a todas as unidades do exército vencedor, sem distinguir o desempenho individual.
- **Tabela de posições:** exércitos com o mesmo nome em batalhas diferentes são tratados como um só, e as estatísticas vêm da primeira instância encontrada.

3 Conclusão

O trabalho resultou em um simulador funcional que aplica de forma consistente os principais conceitos de Programação Orientada a Objetos. A classe abstrata Unidade e seu método virtual puro permitem que Exercito calcule o poder total sem conhecer o tipo concreto de cada unidade, e a composição entre Campanha, Batalha, Exercito e Unidade reflete a estrutura do problema. O uso de herança, polimorfismo, encapsulamento, STL, números aleatórios e manipulação de arquivos consolidou o aprendizado da linguagem.